TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data structures, processing logic, and operational parameters required to build a production-grade ETL solution that ingests customer and order data, implements data quality controls, applies SCD Type 2 logic using Apache Hudi, and produces analytical aggregates.

Scope

This TRD covers:

- Technical specifications for AWS Glue job implementation
- Data source and target configurations with exact S3 paths
- Schema definitions and data type mappings
- Transformation and processing logic for each pipeline stage
- Hudi table configuration for SCD Type 2 implementation
- AWS Glue Data Catalog integration specifications
- Error handling and logging strategies
- Runtime parameters and execution strategies

This TRD does NOT cover:

- AWS IAM role and policy definitions
- Glue job scheduling or orchestration configuration
- Infrastructure-as-Code templates (CloudFormation/Terraform)
- Cost optimization strategies
- Performance tuning beyond standard best practices
- Backup and disaster recovery procedures

Assumptions

1. Data Format Assumptions:

- CSV files use comma (`,`) as delimiter
- CSV files include header row as first line

• Character encoding is UTF-8

• Date fields in Order data are in format YYYY-MM-DD (will attempt to parse other formats if encountered)

2. Data Type Assumptions (not specified in FRD):

• CustId: String

• Name: String

• EmailId: String

• Region: String

• OrderId: String

• ItemName: String

• PricePerUnit: Decimal(10,2)

• Qty: Integer

• Date: Date

• TotalAmount: Decimal(12,2)

• IsActive: Boolean

• StartDate: Timestamp

• EndDate: Timestamp

• OpTs: Timestamp

3. Processing Assumptions:

• "Null" string matching is case-sensitive

• Duplicate detection uses exact match across all columns

• First occurrence of duplicate is retained (based on file read order)

• SCD2 change detection compares all business columns (excludes metadata columns)

• Business key for SCD2 is composite: CustId + OrderId

• Incremental processing uses file modification timestamp to identify new/changed customer files

4. Infrastructure Assumptions:

• AWS Glue Data Catalog database `gen_ai_poc_databrickscoe` exists

• Glue job has IAM permissions for S3 read/write and Glue Catalog operations

• Hudi connector libraries (0.12.x or compatible) are available in Glue environment

• Glue version 4.0 or higher is used (supports Spark 3.3+)

• Sufficient DPU allocation for processing expected data volumes

5. Operational Assumptions:

• Pipeline runs in batch mode (not streaming)

• Initial load processes all historical data

• Subsequent runs process incremental changes

• No concurrent writes to same target tables

• S3 eventual consistency is acceptable for read-after-write operations

6. Catalog Assumptions:

• Parquet format is used for non-Hudi catalog tables

• Hudi format is used for ordersummary table

• Table overwrite mode is used for customer and order catalog tables

• Upsert mode is used for Hudi ordersummary table

• Append mode is used for customeraggregatespend table

•

2. Technical Requirements

TR-INGEST-001: Customer Data Ingestion from S3

• Related Functional Requirement ID(s): FR-INGEST-001

• Objective:

Read customer CSV files from S3 source location into a Spark DataFrame with proper schema inference and validation.

• Source Details:

• Source Type: AWS S3

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV

• Source File Pattern: .csv (all CSV files in directory)

• Header: Yes (first row)

• Delimiter: Comma (,)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_customer_raw

• Input Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	CustId	String	No	Customer unique identifier	
	Name	String	No	Customer full name	

	EmailId	String	No	Customer email address	
	Region	String	No	Customer geographic region	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize Spark session with Glue context
2. Use Glue DynamicFrame or Spark DataFrame reader
3. Configure CSV reader options:
• `header=true`
• `inferSchema=true` (with validation against expected schema)
• `mode=PERMISSIVE` (to capture malformed records)
4. Read all CSV files from source path using wildcard pattern
5. Validate that all expected columns are present
6. Load into DataFrame `df_customer_raw`

• Validation Rules:

• All four columns (CustId, Name, EmailId, Region) must be present
• At least one row must be read (excluding header)
• Column names must match exactly (case-sensitive)
• No schema drift allowed (reject files with additional/missing columns)

• Error Handling and Logging:

• S3 Path Not Found: Log error with full path, raise exception, halt job
• No Files Found: Log warning, create empty DataFrame with schema, continue
• Malformed CSV: Log warning with file name, skip file, continue with remaining files
• Schema Mismatch: Log error with expected vs actual schema, raise exception, halt job
• Read Permission Error: Log error with IAM role details, raise exception, halt job

• Runtime Parameters:

• `source_customer_path`: s3://adif-sdlc/sdlc_wizard/customerdata/
• `csv_delimiter`: ","
• `csv_header`: "true"
• `csv_infer_schema`: "true"

• Operational Notes:

• Use Glue bookmark to track processed files if incremental processing is enabled

• Consider partitioning source data by date if volume grows significantly

• Monitor CloudWatch metrics for read throughput and error rates

•

TR-INGEST-002: Order Data Ingestion from S3

• Related Functional Requirement ID(s): FR-INGEST-002

• Objective:

Read order transaction CSV files from S3 source location into a Spark DataFrame with proper schema inference and validation.

• Source Details:

• Source Type: AWS S3

• Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/

• Source Format: CSV

• Source File Pattern: .csv (all CSV files in directory)

• Header: Yes (first row)

• Delimiter: Comma (,)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_order_raw

• Input Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	OrderId	String	No	Order unique identifier	
	ItemName	String	No	Product/item name	
	PricePerUnit	Decimal(10,2)	No	Unit price of item	
	Qty	Integer	No	Quantity ordered	
	Date	Date	No	Order date	
	CustId	String	No	Customer identifier (FK to Customer)	

• Output Schema:

Same as Input Schema (pass-through at this stage)

• Processing / Transformation Logic:

1. Initialize Spark session with Glue context

2. Use Glue DynamicFrame or Spark DataFrame reader

3. Configure CSV reader options:

• `header=true`

• `inferSchema=true` (with validation against expected schema)

• `mode=PERMISSIVE`

• `dateFormat=yyyy-MM-dd` (primary format, will attempt others)

4. Read all CSV files from source path using wildcard pattern

5. Validate that all expected columns are present

6. Cast Date column to proper Date type if read as String

7. Load into DataFrame `df_order_raw`

• Validation Rules:

• All six columns must be present

• At least one row must be read (excluding header)

• Column names must match exactly (case-sensitive)

• PricePerUnit must be numeric and non-negative

• Qty must be integer and positive

• Date must be parseable to Date type

• CustId must be non-empty string

• Error Handling and Logging:

• S3 Path Not Found: Log error with full path, raise exception, halt job

• No Files Found: Log warning, create empty DataFrame with schema, continue

• Malformed CSV: Log warning with file name, skip file, continue with remaining files

• Schema Mismatch: Log error with expected vs actual schema, raise exception, halt job

• Date Parse Error: Log warning with row details, mark row for exclusion in cleaning stage

• Type Cast Error: Log warning with column and value, mark row for exclusion in cleaning stage

• Runtime Parameters:

• `source_order_path`: s3://adif-sdlc/sdlc_wizard/orderdata/

• `csv_delimiter`: ","

• `csv_header`: "true"

• `csv_infer_schema`: "true"

• `date_format`: "yyyy-MM-dd"

• Operational Notes:

• Use Glue bookmark to track processed files if incremental processing is enabled

• Consider partitioning source data by date if volume grows significantly

• Monitor CloudWatch metrics for read throughput and error rates

• Date parsing may require multiple format attempts (MM/dd/yyyy, dd-MM-yyyy, etc.)

•

TR-CLEAN-001: Customer Data Quality Processing

• Related Functional Requirement ID(s): FR-CLEAN-001

• Objective:

Remove rows containing null values (both string "Null" and actual NULL) and eliminate duplicate records from customer dataset to ensure data quality.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_customer_raw (from TR-INGEST-001)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_customer_clean

• Input Schema:

Same as TR-INGEST-001 output schema

• Output Schema:

Same as Input Schema (subset of rows after cleaning)

• Processing / Transformation Logic:

1. Null Removal:

• For each column (CustId, Name, EmailId, Region):

• Filter out rows where column value is NULL (actual null)

• Filter out rows where column value equals string "Null" (case-sensitive exact match)

• Apply filters using AND logic (row must pass all column checks)

2. Duplicate Removal:

• Identify duplicates based on all columns: (CustId, Name, EmailId, Region)

• Use Spark `dropDuplicates()` function without subset parameter

• Retain first occurrence based on DataFrame row order

• Remove all subsequent duplicate occurrences

3. Result Assignment:

• Assign cleaned DataFrame to `df_customer_clean`

• Validation Rules:

• After null removal, no column should contain NULL or "Null" string

• After deduplication, no two rows should have identical values across all columns

• Row count should be <= input row count

• If all rows are removed, result is empty DataFrame with schema intact

• Error Handling and Logging:

• All Rows Removed: Log warning with count of rows removed by null vs duplicate, continue with empty DataFrame

• No Nulls Found: Log info message, proceed to deduplication

• No Duplicates Found: Log info message, proceed with all rows

• Log summary statistics:

• Input row count

• Rows removed due to nulls

• Rows removed due to duplication

• Output row count

• Runtime Parameters:

• `null_string_value`: "Null" (case-sensitive)

• `dedup_columns`: ["CustId", "Name", "EmailId", "Region"]

• Operational Notes:

• Null removal is performed before deduplication to avoid unnecessary duplicate checks

• Consider adding data quality metrics to CloudWatch for monitoring

• If "Null" string appears frequently, investigate upstream data source issues

•

TR-CLEAN-002: Order Data Quality Processing

• Related Functional Requirement ID(s): FR-CLEAN-002

• Objective:

Remove rows containing null values (both string "Null" and actual NULL) and eliminate duplicate records from order dataset to ensure data quality.

• Source Details:

• Source Type: In-memory Spark DataFrame

• Source Name: df_order_raw (from TR-INGEST-002)

• Target Details:

• Target Type: In-memory Spark DataFrame

• Target Name: df_order_clean

• Input Schema:

Same as TR-INGEST-002 output schema

• Output Schema:

Same as Input Schema (subset of rows after cleaning)

• Processing / Transformation Logic:

1. Null Removal:

• For each column (OrderId, ItemName, PricePerUnit, Qty, Date, CustId):

• Filter out rows where column value is NULL (actual null)

• Filter out rows where column value equals string "Null" (case-sensitive exact match)

• Apply filters using AND logic (row must pass all column checks)

2. Duplicate Removal:

• Identify duplicates based on all columns: (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)

• Use Spark `dropDuplicates()` function without subset parameter

• Retain first occurrence based on DataFrame row order

• Remove all subsequent duplicate occurrences

3. Result Assignment:

• Assign cleaned DataFrame to `df_order_clean`

• Validation Rules:

• After null removal, no column should contain NULL or "Null" string

• After deduplication, no two rows should have identical values across all columns

• Row count should be <= input row count

• If all rows are removed, result is empty DataFrame with schema intact

• PricePerUnit should be non-negative after cleaning

• Qty should be positive integer after cleaning

• Error Handling and Logging:

• All Rows Removed: Log warning with count of rows removed by null vs duplicate, continue with empty DataFrame

• No Nulls Found: Log info message, proceed to deduplication

• No Duplicates Found: Log info message, proceed with all rows

- Log summary statistics:
- Input row count
- Rows removed due to nulls
- Rows removed due to duplication
- Output row count
- Min/Max/Avg PricePerUnit
- Min/Max/Avg Qty

- Runtime Parameters:
- `null_string_value`: "Null" (case-sensitive)
- `dedup_columns`: ["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"]

- Operational Notes:
- Null removal is performed before deduplication to avoid unnecessary duplicate checks
- Consider adding data quality metrics to CloudWatch for monitoring
- If "Null" string appears frequently, investigate upstream data source issues
- Monitor for negative PricePerUnit or zero/negative Qty values

-

TR-CATALOG-001: Customer Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001

- Objective:

Write cleaned customer data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_customer_clean (from TR-CLEAN-001)

- Target Details:
- Target Type: AWS S3 + Glue Data Catalog
- Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/customer/
- Target Format: Parquet
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_customer
- Write Mode: Overwrite

• Input Schema:

Same as TR-CLEAN-001 output schema

• Output Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	CustId	String	No	Customer unique identifier	
	Name	String	No	Customer full name	
	EmailId	String	No	Customer email address	
	Region	String	No	Customer geographic region	

• Processing / Transformation Logic:

1. Take `df_customer_clean` DataFrame

2. Write to S3 using Spark DataFrameWriter:

• Format: Parquet

• Mode: Overwrite (replace existing data)

• Compression: Snappy (default)

• Path: s3://adif-sdlc/curated/sdlc_wizard/customer/

3. Register table in Glue Data Catalog:

• Database: gen_ai_poc_databrickscoe

• Table: sdlc_wizard_customer

• Location: s3://adif-sdlc/curated/sdlc_wizard/customer/

• Format: Parquet

• Schema: Infer from DataFrame or explicitly define

• Validation Rules:

• S3 write must complete successfully before catalog registration

• Catalog database must exist before table creation

• Table schema in catalog must match DataFrame schema

• At least one Parquet file must be written to S3

• Error Handling and Logging:

• S3 Write Failure: Log error with S3 path and exception details, raise exception, halt job

• Database Not Found: Log error with database name, raise exception, halt job

• Catalog Registration Failure: Log error with table details, raise exception, halt job

• Permission Error: Log error with IAM role and resource ARN, raise exception, halt job

• Log success message with:

• Row count written

- S3 path
- Catalog database.table name
- File size and count

- Runtime Parameters:
- `target_customer_s3_path`: s3://adif-sdlc/curated/sdlc_wizard/customer/
- `catalog_database`: gen_ai_poc_databrickscoe
- `catalog_table_customer`: sdlc_wizard_customer
- `write_mode`: overwrite
- `file_format`: parquet
- `compression`: snappy

- Operational Notes:
- Overwrite mode will delete existing data in S3 path
- Consider partitioning by Region if data volume grows significantly
- Parquet format provides efficient columnar storage and compression
- Table is immediately queryable via Athena after registration
- Monitor S3 storage costs and file count

-

TR-CATALOG-002: Order Table Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-002

- Objective:

Write cleaned order data to S3 in Parquet format and register as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_order_clean (from TR-CLEAN-002)

- Target Details:
- Target Type: AWS S3 + Glue Data Catalog
- Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/order/
- Target Format: Parquet
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_order
- Write Mode: Overwrite

• Input Schema:

Same as TR-CLEAN-002 output schema

• Output Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	OrderId	String	No	Order unique identifier	
	ItemName	String	No	Product/item name	
	PricePerUnit	Decimal(10,2)	No	Unit price of item	
	Qty	Integer	No	Quantity ordered	
	Date	Date	No	Order date	
	CustId	String	No	Customer identifier (FK)	

• Processing / Transformation Logic:

1. Take `df_order_clean` DataFrame

2. Write to S3 using Spark DataFrameWriter:

• Format: Parquet

• Mode: Overwrite (replace existing data)

• Compression: Snappy (default)

• Path: s3://adif-sdlc/curated/sdlc_wizard/order/

3. Register table in Glue Data Catalog:

• Database: gen_ai_poc_databrickscoe

• Table: sdlc_wizard_order

• Location: s3://adif-sdlc/curated/sdlc_wizard/order/

• Format: Parquet

• Schema: Infer from DataFrame or explicitly define

• Validation Rules:

• S3 write must complete successfully before catalog registration

• Catalog database must exist before table creation

• Table schema in catalog must match DataFrame schema

• At least one Parquet file must be written to S3

• Date column must be properly formatted as Date type

• Error Handling and Logging:

• S3 Write Failure: Log error with S3 path and exception details, raise exception, halt job

• Database Not Found: Log error with database name, raise exception, halt job

• Catalog Registration Failure: Log error with table details, raise exception, halt job

• Permission Error: Log error with IAM role and resource ARN, raise exception, halt job

• Log success message with:

• Row count written

• S3 path

• Catalog database.table name

• File size and count

• Date range of orders

• Runtime Parameters:

• `target_order_s3_path`: s3://adif-sdlc/curated/sdlc_wizard/order/

• `catalog_database`: gen_ai_poc_databrickscoe

• `catalog_table_order`: sdlc_wizard_order

• `write_mode`: overwrite

• `file_format`: parquet

• `compression`: snappy

• Operational Notes:

• Overwrite mode will delete existing data in S3 path

• Consider partitioning by Date if data volume grows significantly

• Parquet format provides efficient columnar storage and compression

• Table is immediately queryable via Athena after registration

• Monitor S3 storage costs and file count

•

TR-SCD2-001: Order Summary SCD Type 2 Implementation

• Related Functional Requirement ID(s): FR-SCD2-001, FR-CATALOG-003

• Objective:

Join customer and order data, implement Slowly Changing Dimension Type 2 logic to track historical changes, and store results in a Hudi table on S3 with Glue Catalog registration.

• Source Details:

• Source Type: In-memory Spark DataFrames

• Source Names:

• df_customer_clean (from TR-CLEAN-001)

• df_order_clean (from TR-CLEAN-002)

• Join Type: Inner Join

• Join Key: CustId

• Target Details:

• Target Type: Apache Hudi Table on S3 + Glue Data Catalog

• Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Target Format: Hudi (Copy-on-Write)

• Target Database: gen_ai_poc_databrickscoe

• Target Table: ordersummary

• Write Mode: Upsert (for SCD2 logic)

• Hudi Table Type: COPY_ON_WRITE

• Record Key: CustId, OrderId (composite key)

• Precombine Key: OpTs (operation timestamp)

• Partition Path: None (or Date if specified later)

• Input Schema:

• Customer: CustId, Name, EmailId, Region

• Order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Output Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	CustId	String	No	Customer identifier (part of record key)	
	OrderId	String	No	Order identifier (part of record key)	
	Name	String	No	Customer name	
	EmailId	String	No	Customer email	
	Region	String	No	Customer region	
	ItemName	String	No	Product/item name	
	PricePerUnit	Decimal(10,2)	No	Unit price	
	Qty	Integer	No	Quantity ordered	
	Date	Date	No	Order date	
	IsActive	Boolean	No	SCD2 active flag (true=current, false=historical)	
	StartDate	Timestamp	No	SCD2 effective start timestamp	
	EndDate	Timestamp	Yes	SCD2 effective end timestamp (null for active records)	
	OpTs	Timestamp	No	Operation timestamp (for Hudi precombine)	

• Processing / Transformation Logic:

1. Initial Join:

```
df_joined = df_customer_clean.join(df_order_clean, on="CustId", how="inner")
```

2. Add SCD2 Metadata Columns:

• Add `IsActive = true` for all new records

• Add `StartDate = current_timestamp()`

• Add `EndDate = null`

• Add `OpTs = current_timestamp()`

3. SCD2 Logic Implementation:

• For Initial Load (target table does not exist):

• Write all joined records with SCD2 metadata to Hudi table

• All records have IsActive=true, EndDate=null

• For Incremental Load (target table exists):

• Read existing Hudi table into DataFrame `df_existing`

• For each record in `df_joined`:

• Create business key: (CustId, OrderId)

• Check if business key exists in `df_existing`

• Case 1: New Record (business key not found)

• Insert new record with IsActive=true, StartDate=current_timestamp(), EndDate=null, OpTs=current_timestamp()

• Case 2: Existing Record - No Change

• Compare all business columns (Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date)

• If all match: No action required

• Case 3: Existing Record - Changed

• Compare all business columns

• If any differ:

• Update existing active record: Set IsActive=false, EndDate=current_timestamp(), OpTs=current_timestamp()

• Insert new record: IsActive=true, StartDate=current_timestamp(), EndDate=null, OpTs=current_timestamp(), with new values

4. Hudi Write Configuration:

```
```

```
hudi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'CustId,OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.operation': 'upsert',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.upsert.shuffle.parallelism': 2,
'hoodie.insert.shuffle.parallelism': 2
}
```

5. Write to Hudi:

• Use Spark DataFrameWriter with Hudi format

• Apply SCD2 logic through custom merge logic or Hudi upsert

• Write to s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

6. Catalog Registration:

• Register Hudi table in Glue Data Catalog

• Database: gen_ai_poc_databrickscoe

• Table: ordersummary

• Enable Hudi metadata for Athena queries

• Validation Rules:

• Join must produce at least one record (unless both sources are empty)

• No duplicate business keys (CustId, OrderId) with IsActive=true

• All active records must have EndDate=null

• All inactive records must have EndDate populated

• StartDate must be <= EndDate for inactive records

• OpTs must be populated for all records

• For any business key, only one record should have IsActive=true

• Error Handling and Logging:

• Join Produces No Records: Log warning, create empty Hudi table with schema, continue

• Duplicate Active Records: Log error with business key details, raise exception, halt job

• Hudi Write Failure: Log error with exception details, raise exception, halt job

• Catalog Registration Failure: Log error, raise exception, halt job

• Log summary statistics:

- Input customer count
- Input order count
- Joined record count
- New records inserted
- Records updated (closed + new version)
- Records unchanged
- Total active records
- Total historical records

- Runtime Parameters:
- `target_ordersummary_s3_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- `catalog_database`: gen_ai_poc_databrickscoe
- `catalog_table_ordersummary`: ordersummary
- `hudi_table_name`: ordersummary
- `hudi_record_key`: CustId,OrderId
- `hudi_precombine_key`: OpTs
- `hudi_operation`: upsert
- `hudi_table_type`: COPY_ON_WRITE
- `is_initial_load`: true/false (determines SCD2 logic path)

- Operational Notes:
- Initial load should set `is_initial_load=true` to skip SCD2 comparison logic
- Subsequent runs should set `is_initial_load=false` to enable SCD2 processing
- Hudi COPY_ON_WRITE provides snapshot isolation and better read performance
- Consider partitioning by Date if data volume grows significantly
- Monitor Hudi compaction and cleaning operations
- Athena queries require Hudi connector for proper read support
- OpTs is critical for Hudi's precombine logic during upserts

-

TR-INCR-001: Incremental Customer Update Processing

- Related Functional Requirement ID(s): FR-INCR-001

- Objective:

Detect new or changed customer records and trigger SCD Type 2 updates in the order summary table incrementally, avoiding full reprocessing.

• Source Details:

• Source Type: AWS S3 (new/modified files)

• Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/

• Source Format: CSV

• Change Detection Method: File modification timestamp or Glue bookmark

• Target Details:

• Target Type: Apache Hudi Table (existing)

• Target Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

• Update Mode: Incremental SCD2 upsert

• Input Schema:

• New/Changed Customer: CustId, Name, EmailId, Region

• Existing Order Summary: Full schema from TR-SCD2-001

• Output Schema:

Same as TR-SCD2-001 (updated ordersummary table)

• Processing / Transformation Logic:

1. Identify Changed Customers:

• Enable Glue job bookmark to track processed files

• Read only new/modified customer CSV files since last run

• Apply same cleaning logic as TR-CLEAN-001

• Result: `df_customer_changed`

2. Retrieve Affected Orders:

• Read cleaned order data (from catalog or S3)

• Filter orders where CustId IN (changed customer CustIds)

• Result: `df_order_affected`

3. Apply Incremental SCD2 Logic:

• Join `df_customer_changed` with `df_order_affected` on CustId

• Read existing ordersummary Hudi table

• For each joined record:

• Create business key: (CustId, OrderId)

• Find matching record in existing ordersummary where IsActive=true

• Case 1: No Active Record Found (New Order)

• Insert new record with IsActive=true, StartDate=current_timestamp(), EndDate=null, OpTs=current_timestamp()

• Case 2: Active Record Found - Customer Data Changed

• Compare customer columns (Name, EmailId, Region)

• If changed:

• Close existing record: Set IsActive=false, EndDate=current_timestamp(), OpTs=current_timestamp()

• Insert new record: IsActive=true, StartDate=current_timestamp(), EndDate=null, OpTs=current_timestamp(), with updated customer data

• Case 3: Active Record Found - No Change

• No action required

4. Hudi Incremental Upsert:

• Use same Hudi configuration as TR-SCD2-001

• Write mode: upsert

• Hudi will handle merge logic based on record key and precombine key

5. Update OpTs:

• Set OpTs = current_timestamp() for all modified records

• Ensures proper ordering in Hudi precombine logic

• Validation Rules:

• At least one changed customer must be detected (or skip processing)

• All affected orders must be retrieved

• No duplicate active records after update

• All closed records must have EndDate populated

• OpTs must be monotonically increasing for same business key

• Error Handling and Logging:

• No Changed Customers: Log info message "No customer changes detected", skip processing, exit successfully

• No Affected Orders: Log warning "Changed customers have no orders", skip SCD2 update, continue

• Hudi Upsert Failure: Log error with exception details, raise exception, halt job

• Bookmark Update Failure: Log error, raise exception (to prevent reprocessing same files)

• Log summary statistics:

• Changed customer count

• Affected order count

• Records closed (made inactive)
• New records inserted
• Records unchanged
• Processing duration

• Runtime Parameters:
• `source_customer_path`: s3://adif-sdlc/sdlc_wizard/customerdata/
• `enable_bookmark`: true
• `bookmark_key`: customer_incremental_load
• `target_ordersummary_s3_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
• `incremental_mode`: true

• Operational Notes:
• This requirement assumes Glue job bookmark is enabled for incremental processing
• Alternative: Use watermark column (e.g., LastModifiedDate) in customer data
• First run should process all customers (initial load via TR-SCD2-001)
• Subsequent runs process only new/changed files
• Consider scheduling this job more frequently than full refresh
• Monitor bookmark state in Glue console
• If bookmark is reset, job will reprocess all files (may cause duplicate SCD2 versions)
•

TR-AGG-001: Customer Aggregate Spend Calculation

• Related Functional Requirement ID(s): FR-AGG-001, FR-CATALOG-004

• Objective:

Aggregate order data by customer and date to calculate total spending, and store results in an analytics table for business intelligence.

• Source Details:
• Source Type: Apache Hudi Table
• Source Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
• Source Table: gen_ai_poc_databrickscoe.ordersummary
• Filter Condition: IsActive = true (only current records)

• Target Details:
• Target Type: AWS S3 + Glue Data Catalog
• Target S3 Path: s3://adif-sdlc/analytics/customeraggregatespend/

• Target Format: Parquet

• Target Database: gen_ai_poc_databrickscoe

• Target Table: customeraggregatespend

• Write Mode: Append (or Overwrite based on operational strategy)

• Input Schema:

Source columns from ordersummary (filtered for IsActive=true):

• Name (String)

• PricePerUnit (Decimal(10,2))

• Qty (Integer)

• Date (Date)

• IsActive (Boolean) - used for filtering only

• Output Schema:

	Column Name	Data Type	Nullable	Description	
	---	---	---	---	
	Name	String	No	Customer name	
	TotalAmount	Decimal(12,2)	No	Sum of (PricePerUnit * Qty) for customer on date	
	Date	Date	No	Order date	

• Processing / Transformation Logic:

1. Read Active Order Summary Records:

```
df_ordersummary = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
df_active = df_ordersummary.filter(col("IsActive") == true)
```

2. Calculate TotalAmount:

```
df_with_total = df_active.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))
```

3. Group and Aggregate:

```
df_aggregated = df_with_total.groupBy("Name", "Date").agg(
sum("TotalAmount").alias("TotalAmount")
)
```

4. Select Final Columns:

```
df_final = df_aggregated.select("Name", "TotalAmount", "Date")
```

5. Write to S3:

• Format: Parquet

• Mode: Append (to accumulate historical aggregates) or Overwrite (for full refresh)

• Path: s3://adif-sdlc/analytics/customeraggregatespend/

6. Register in Glue Catalog:

• Database: gen_ai_poc_databrickscoe

• Table: customeraggregatespend

• Location: s3://adif-sdlc/analytics/customeraggregatespend/

• Format: Parquet

• Validation Rules:

• TotalAmount must be non-negative (PricePerUnit and Qty are non-negative)

• Each (Name, Date) combination should appear only once in output

• At least one record should be produced (unless ordersummary is empty)

• TotalAmount precision should not overflow Decimal(12,2)

• Error Handling and Logging:

• Empty Order Summary: Log warning "No active orders found", create empty aggregate table with schema, continue

• Aggregation Failure: Log error with exception details, raise exception, halt job

• S3 Write Failure: Log error with S3 path and exception, raise exception, halt job

• Catalog Registration Failure: Log error with table details, raise exception, halt job

• Decimal Overflow: Log error with affected records, raise exception, halt job

• Log summary statistics:

• Input record count (active orders)

• Output record count (aggregated rows)

• Total amount sum across all customers

• Unique customer count

• Date range covered

• Min/Max/Avg TotalAmount per customer-date

• Runtime Parameters:

- `source_ordersummary_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- `target_aggregate_s3_path`: s3://adif-sdlc/analytics/customeraggregatespend/
- `catalog_database`: gen_ai_poc_databrickscoe
- `catalog_table_aggregate`: customeraggregatespend
- `write_mode`: append (or overwrite)
- `file_format`: parquet
- `compression`: snappy

- Operational Notes:
- Append mode allows incremental aggregation if ordersummary is updated incrementally
- Overwrite mode provides full refresh of aggregates (simpler but less efficient)
- Consider partitioning by Date if data volume grows significantly
- TotalAmount calculation (PricePerUnit Qty) may require rounding to 2 decimal places
- Monitor for potential data skew if few customers have many orders
- This table is optimized for BI tools and Athena queries
- Consider creating additional aggregations (by Region, by Month, etc.) as separate requirements
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: PySpark (Python with Spark DataFrame API)
- Rationale:
- Native support in AWS Glue
- Rich ecosystem for data processing
- Better readability and maintainability than Scala for this use case
- Extensive AWS SDK integration

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Glue Version: 4.0 (Spark 3.3.0, Python 3.10)
- Worker Type: G.1X or G.2X (based on data volume)
- Number of Workers: Start with 2-5, scale based on performance testing
- Job Timeout: 2880 minutes (48 hours) - adjust based on data volume
- Max Retries: 1 (to handle transient failures)
- Max Concurrent Runs: 1 (to prevent concurrent writes to same tables)

Glue Job Type Expectations

• Script Type: Python Shell or Spark (Spark recommended for data volume and transformations)

• Execution Mode: Batch (not streaming)

• Bookmark: Enabled for incremental processing (TR-INCR-001)

• Job Parameters:

• `--enable-glue-datacatalog`: true

• `--enable-metrics`: true

• `--enable-continuous-cloudwatch-log`: true

• `--enable-spark-ui`: true

• `--spark-event-logs-path`: s3:///spark-logs/

• `--TempDir`: s3:///temp/

• `--enable-job-insights`: true

Dependency Management

• Hudi Libraries:

• Use Glue's built-in Hudi support (version 0.12.x)

• Or specify via `--datalake-formats`: hudi

• Additional Python Libraries: None required beyond Glue defaults

• Spark Configuration:

• `spark.serializer`: org.apache.spark.serializer.KryoSerializer

• `spark.sql.hive.convertMetastoreParquet`: false (for Hudi compatibility)

Orchestration Considerations

• Initial Load Sequence:

1. TR-INGEST-001 (Customer ingestion)
2. TR-INGEST-002 (Order ingestion)
3. TR-CLEAN-001 (Customer cleaning)
4. TR-CLEAN-002 (Order cleaning)
5. TR-CATALOG-001 (Customer catalog)
6. TR-CATALOG-002 (Order catalog)
7. TR-SCD2-001 (Order summary with SCD2 - initial load)
8. TR-AGG-001 (Customer aggregate spend)

• Incremental Load Sequence:

1. TR-INCR-001 (Incremental customer updates)
2. TR-AGG-001 (Refresh customer aggregate spend)

• Recommended Orchestration: AWS Step Functions or AWS Glue Workflows

• Scheduling: AWS EventBridge (CloudWatch Events) for time-based triggers

•

4. Data Design Details

Source Systems and Paths

	Source Name	Source Type	Source Path	Format	Schema Reference	
	---	---	---	---	---	
	Customer Data	AWS S3	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	TR-INGEST-001 Input Schema	
	Order Data	AWS S3	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	TR-INGEST-002 Input Schema	

• Note: These are the ORIGINAL sources as specified in FRD. No intermediate staging tables or alternative sources are introduced.

Target Systems and Paths

	Target Type	Target Path	Format	Schema Reference	U
	---	---	---	---	-
	S3 + Glue Catalog	s3://adif-sdlc/curated/sdlc_wizard/customer/	Parquet	TR-CATALOG-001 Output Schema	C
	S3 + Glue Catalog	s3://adif-sdlc/curated/sdlc_wizard/order/	Parquet	TR-CATALOG-002 Output Schema	C
	S3 + Glue Catalog (Hudi)	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi (COW)	TR-SCD2-001 Output Schema	U
e Spend	S3 + Glue Catalog	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	TR-AGG-001 Output Schema	A

Catalog Registration Details

• Database Name: gen_ai_poc_databrickscoe (must exist before job execution)

• Tables:

• sdlc_wizard_customer (Parquet)

• sdlc_wizard_order (Parquet)

• ordersummary (Hudi)

• customeraggregatespend (Parquet)

• Catalog Update Mode: Automatic via Glue DynamicFrame or explicit via Glue API

• Athena Compatibility: All tables must be queryable via Athena

Partitioning Strategy

• Current State: No partitioning specified in FRD

• Recommendations for Future:

• Customer: Partition by Region (if data volume grows)

• Order: Partition by Date (year/month/day)

• Order Summary: Partition by Date (year/month/day)

• Customer Aggregate Spend: Partition by Date (year/month)

File Formats and Compression

	Table	Format	Compression	Rationale	
	---	---	---	---	
	Customer	Parquet	Snappy	Efficient columnar storage, good compression ratio	
	Order	Parquet	Snappy	Efficient columnar storage, good compression ratio	
	Order Summary	Hudi (Parquet base)	Snappy	SCD2 support, ACID transactions, time travel queries	
	Customer Aggregate Spend	Parquet	Snappy	Optimized for analytical queries	

Update/Overwrite/Append Behavior

• Customer Catalog (sdlc_wizard_customer): Overwrite - full refresh on each run

• Order Catalog (sdlc_wizard_order): Overwrite - full refresh on each run

• Order Summary (ordersummary): Upsert - SCD2 logic with Hudi merge

• Customer Aggregate Spend (customeraggregatespend): Append (incremental) or Overwrite (full refresh) - configurable

Data Retention and Lifecycle

• Not specified in FRD - Assumptions:

• Retain all historical SCD2 records indefinitely in ordersummary

• Apply S3 lifecycle policies for cost optimization (e.g., move to Glacier after 1 year)

• Hudi cleaning and compaction policies to manage file count

•

5. Processing Details

Data Quality Rules

Null Handling

• String "Null": Treat as missing value, remove row

• Actual NULL: Treat as missing value, remove row

• Case Sensitivity: "Null" is case-sensitive (exact match)

• Application: Apply to all columns in customer and order datasets before any processing

Deduplication Rules

• Scope: All columns in each dataset

• Method: Exact match across all columns

• Retention: Keep first occurrence based on DataFrame row order

• Timing: Apply after null removal, before catalog registration

Data Type Validation

- PricePerUnit: Must be numeric, non-negative, max 2 decimal places
- Qty: Must be integer, positive (> 0)
- Date: Must be parseable to Date type, valid date range
- CustId, OrderId: Must be non-empty strings
- EmailId: Must be non-empty string (format validation not specified)

Referential Integrity

- Order.CustId -> Customer.CustId: Enforced via inner join in TR-SCD2-001
- Orphaned Orders: Excluded from order summary (inner join behavior)
- Missing Customers: Orders without matching customers are dropped

Join Rules

Customer-Order Join (TR-SCD2-001)

- Join Type: Inner Join
- Join Key: CustId
- Join Condition: customer.CustId = order.CustId
- Result: Only orders with matching customers are included
- Cardinality: One customer to many orders (1:N)
- Null Handling: Join keys are non-null after cleaning, no special handling needed

Aggregation Rules

Customer Aggregate Spend (TR-AGG-001)

- Grouping Columns: Name, Date
- Aggregation Function: SUM(PricePerUnit Qty) AS TotalAmount
- Filter Condition: IsActive = true (only current SCD2 records)
- Null Handling: No nulls expected after cleaning and SCD2 processing
- Precision: TotalAmount as Decimal(12,2) to accommodate large sums

SCD Type 2 / History Tracking Rules

SCD2 Metadata Columns

- IsActive (Boolean):
- true = current/active record
- false = historical/closed record
- Only one active record per business key at any time
- StartDate (Timestamp):

• Effective start date/time of the record

• Set to current_timestamp() when record is created

• EndDate (Timestamp):

• Effective end date/time of the record

• NULL for active records

• Set to current_timestamp() when record is closed

• OpTs (Timestamp):

• Operation timestamp for Hudi precombine logic

• Set to current_timestamp() for all inserts and updates

• Used by Hudi to determine latest version during upsert

Business Key Definition

• Composite Key: (CustId, OrderId)

• Uniqueness: Each combination of CustId and OrderId represents a unique business entity

• Immutability: Business key values do not change once assigned

Change Detection Logic

• Comparison Columns: All business columns (Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date)

• Change Trigger: Any difference in comparison columns triggers SCD2 update

• Comparison Method: Exact match (case-sensitive for strings, exact value for numerics/dates)

SCD2 State Transitions

1. New Record: Insert with IsActive=true, StartDate=now, EndDate=null, OpTs=now

2. No Change: No action, existing active record remains unchanged

3. Change Detected:

• Close existing active record: IsActive=false, EndDate=now, OpTs=now

• Insert new active record: IsActive=true, StartDate=now, EndDate=null, OpTs=now, with updated values

Hudi-Specific SCD2 Implementation

• Upsert Operation: Hudi upsert with custom merge logic

• Precombine Field: OpTs (ensures latest operation wins)

• Record Key: CustId,OrderId (composite key)

• Merge Logic:

• If record key exists and values differ: close old, insert new

• If record key exists and values match: no action

• If record key does not exist: insert new

Type Casting Expectations

Source Data Type Inference

- CSV Inference: Use Spark's inferSchema=true with validation
- Fallback: Explicit schema definition if inference fails

Target Data Type Mapping

	Source Column	Source Type (Inferred)	Target Type	Casting Logic	
	---	---	---	---	
	CustId	String	String	Direct map	
	Name	String	String	Direct map	
	EmailId	String	String	Direct map	
	Region	String	String	Direct map	
	OrderId	String	String	Direct map	
	ItemName	String	String	Direct map	
	PricePerUnit	String/Double	Decimal(10,2)	Cast to Decimal, validate non-negative	
	Qty	String/Integer	Integer	Cast to Integer, validate positive	
	Date	String	Date	Parse with dateFormat, validate range	
	TotalAmount	Calculated	Decimal(12,2)	PricePerUnit * Qty, cast to Decimal(12,2)	
	IsActive	N/A	Boolean	Literal true/false	
	StartDate	N/A	Timestamp	current_timestamp()	
	EndDate	N/A	Timestamp	current_timestamp() or null	
	OpTs	N/A	Timestamp	current_timestamp()	

Date Parsing Strategy

- Primary Format: yyyy-MM-dd
- Fallback Formats: MM/dd/yyyy, dd-MM-yyyy, yyyy/MM/dd
- Error Handling: Log warning and exclude row if all formats fail

Decimal Precision Handling

- PricePerUnit: Decimal(10,2) - max 8 digits before decimal, 2 after
- TotalAmount: Decimal(12,2) - max 10 digits before decimal, 2 after
- Overflow: Raise error if precision exceeded
-

6. Traceability Matrix

RD Requirement ID(s)	Original Source	Technical Source	Description
---	---	---	---

R-INGEST-001	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	Customer CS
R-INGEST-002	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	Order CSV in
R-CLEAN-001	df_customer_raw	df_customer_raw	Customer data
R-CLEAN-002	df_order_raw	df_order_raw	Order data qu
R-CATALOG-001	df_customer_clean	df_customer_clean	Customer cata
R-CATALOG-002	df_order_clean	df_order_clean	Order catalog
R-SCD2-001	df_customer_clean + df_order_clean	df_customer_clean + df_order_clean	Order summa
R-INCR-001	s3://adif-sdlc/sdlc_wizard/customerdata/ (new files)	s3://adif-sdlc/sdlc_wizard/customerdata/ (new files)	Incremental c
R-AGG-001	ordersummary table (IsActive=true)	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Customer agg
R-SCD2-001	ordersummary Hudi table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Order summa
R-AGG-001	customeraggregatespend table	s3://adif-sdlc/analytics/customeraggregatespend/	Aggregate spe

• Note on Source Fidelity: All technical sources in this TRD exactly match the original sources specified in the FRD. No intermediate staging tables, derived sources, or alternative sources have been introduced. The source-to-target lineage is preserved throughout the document.

•

7. ETL Mapping Details

TR-INGEST-001: Customer Data Ingestion Mapping

• Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV files)

• Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Description	
	---	---	---	---	---	
	CustId	String	Variable	No	Customer unique identifier	
	Name	String	Variable	No	Customer full name	
	EmailId	String	Variable	No	Customer email address	
	Region	String	Variable	No	Customer geographic region	

• Target: df_customer_raw (In-memory DataFrame)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	CustId	String	Variable	No	Direct map from source CustId	
	Name	String	Variable	No	Direct map from source Name	
	EmailId	String	Variable	No	Direct map from source EmailId	
	Region	String	Variable	No	Direct map from source Region	

• Transformation Rules:

• All columns: Direct pass-through, no transformation

•

TR-INGEST-002: Order Data Ingestion Mapping

• Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV files)

• Source Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Description	
	---	---	---	---	---	
	OrderId	String	Variable	No	Order unique identifier	
	ItemName	String	Variable	No	Product/item name	
	PricePerUnit	Decimal	10,2	No	Unit price of item	
	Qty	Integer	-	No	Quantity ordered	
	Date	Date	-	No	Order date	
	CustId	String	Variable	No	Customer identifier (FK)	

• Target: df_order_raw (In-memory DataFrame)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	OrderId	String	Variable	No	Direct map from source OrderId	
	ItemName	String	Variable	No	Direct map from source ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from source PricePerUnit, cast to Decimal(10,2)	
	Qty	Integer	-	No	Direct map from source Qty, cast to Integer	
	Date	Date	-	No	Direct map from source Date, parse to Date type	
	CustId	String	Variable	No	Direct map from source CustId	

• Transformation Rules:

• OrderId, ItemName, CustId: Direct pass-through

• PricePerUnit: Cast to Decimal(10,2) if read as String

• Qty: Cast to Integer if read as String

• Date: Parse to Date type using dateFormat=yyyy-MM-dd

•

TR-CLEAN-001: Customer Data Cleaning Mapping

• Source: df_customer_raw (from TR-INGEST-001)

• Source Schema: Same as TR-INGEST-001 target schema

• Target: df_customer_clean (In-memory DataFrame)

• Target Schema: Same as source schema (subset of rows)

• Transformation Rules:
• All columns: Direct pass-through after filtering
• Row-level filtering:
• Remove rows where any column is NULL
• Remove rows where any column equals "Null" (string)
• Remove duplicate rows (all columns match)

•

TR-CLEAN-002: Order Data Cleaning Mapping

• Source: df_order_raw (from TR-INGEST-002)

• Source Schema: Same as TR-INGEST-002 target schema

• Target: df_order_clean (In-memory DataFrame)

• Target Schema: Same as source schema (subset of rows)

• Transformation Rules:
• All columns: Direct pass-through after filtering
• Row-level filtering:
• Remove rows where any column is NULL
• Remove rows where any column equals "Null" (string)
• Remove duplicate rows (all columns match)

•

TR-CATALOG-001: Customer Catalog Registration Mapping

• Source: df_customer_clean (from TR-CLEAN-001)

• Source Schema: Same as TR-CLEAN-001 target schema

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_customer (Glue Catalog + S3 Parquet)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	

	CustId	String	Variable	No	Direct map from source CustId	
	Name	String	Variable	No	Direct map from source Name	
	EmailId	String	Variable	No	Direct map from source EmailId	
	Region	String	Variable	No	Direct map from source Region	

• Transformation Rules:

• All columns: Direct pass-through, write to Parquet format

•

TR-CATALOG-002: Order Catalog Registration Mapping

• Source: df_order_clean (from TR-CLEAN-002)

• Source Schema: Same as TR-CLEAN-002 target schema

• Target: gen_ai_poc_databrickscoe.sdlc_wizard_order (Glue Catalog + S3 Parquet)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	OrderId	String	Variable	No	Direct map from source OrderId	
	ItemName	String	Variable	No	Direct map from source ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from source PricePerUnit	
	Qty	Integer	-	No	Direct map from source Qty	
	Date	Date	-	No	Direct map from source Date	
	CustId	String	Variable	No	Direct map from source CustId	

• Transformation Rules:

• All columns: Direct pass-through, write to Parquet format

•

TR-SCD2-001: Order Summary SCD2 Mapping

• Source 1: df_customer_clean (from TR-CLEAN-001)

• Source 1 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	

	Region	String	Variable	No	

• Source 2: df_order_clean (from TR-CLEAN-002)

• Source 2 Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	---	---	---	---	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Target: gen_ai_poc_databrickscoe.ordersummary (Glue Catalog + S3 Hudi)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	CustId	String	Variable	No	Direct map from customer.CustId (join key)	
	OrderId	String	Variable	No	Direct map from order.OrderId	
	Name	String	Variable	No	Direct map from customer.Name	
	EmailId	String	Variable	No	Direct map from customer.EmailId	
	Region	String	Variable	No	Direct map from customer.Region	
	ItemName	String	Variable	No	Direct map from order.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from order.PricePerUnit	
	Qty	Integer	-	No	Direct map from order.Qty	
	Date	Date	-	No	Direct map from order.Date	
	IsActive	Boolean	-	No	Derived: true for new/updated records, false for closed records	
	StartDate	Timestamp	-	No	Derived: current_timestamp() when record is created	
	EndDate	Timestamp	-	Yes	Derived: null for active records, current_timestamp() when closed	
	OpTs	Timestamp	-	No	Derived: current_timestamp() for all operations	

• Transformation Rules:

• Join customer and order on CustId (inner join)

• Business columns (CustId through Date): Direct map from joined sources

• IsActive: Set to true for new/current records, false for historical records

• StartDate: Set to current_timestamp() when record is inserted

• EndDate: Set to null for active records, current_timestamp() when record is closed

• OpTs: Set to current_timestamp() for all insert/update operations

• SCD2 Logic: Compare all business columns to detect changes, close old record and insert new if changed

•

TR-INCR-001: Incremental Customer Update Mapping

• Source 1: s3://adif-sdlc/sdlc_wizard/customerdata/ (new/modified CSV files)

• Source 1 Schema: Same as TR-INGEST-001 source schema

• Source 2: gen_ai_poc_databrickscoe.ordersummary (existing Hudi table)

• Source 2 Schema: Same as TR-SCD2-001 target schema

• Target: gen_ai_poc_databrickscoe.ordersummary (updated Hudi table)

• Target Schema: Same as TR-SCD2-001 target schema

• Transformation Rules:

• Read only new/changed customer files (via Glue bookmark)

• Apply same cleaning logic as TR-CLEAN-001

• Filter orders for changed customers

• Apply SCD2 logic as in TR-SCD2-001 (incremental mode)

• Update OpTs to current_timestamp() for all modified records

•

TR-AGG-001: Customer Aggregate Spend Mapping

• Source: gen_ai_poc_databrickscoe.ordersummary (Hudi table, IsActive=true)

• Source Schema: Same as TR-SCD2-001 target schema (filtered for IsActive=true)

• Target: gen_ai_poc_databrickscoe.customeraggregatespend (Glue Catalog + S3 Parquet)

• Target Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	---	---	---	---	---	
	Name	String	Variable	No	Direct map from source Name (grouping column)	
	TotalAmount	Decimal	12,2	No	Derived: SUM(PricePerUnit * Qty) grouped by Name and Date	
	Date	Date	-	No	Direct map from source Date (grouping column)	

• Transformation Rules:

• Filter source for IsActive = true

• Calculate TotalAmount = PricePerUnit Qty for each row

• Group by Name and Date

• Aggregate: SUM(TotalAmount) AS TotalAmount

• Select final columns: Name, TotalAmount, Date

•

• End of Technical Requirements Document

•

• Note: Not enough information available in the FRD to derive exact string lengths for variable-length string columns (CustId, Name, EmailId, Region, OrderId, ItemName). These are specified as "Variable" and will be determined by actual data or can be constrained in implementation if business rules are provided.